

Monte Carlo Tree Search for Tic-Tac-Toe

A step-by-step guide to implementing MCTS as a `strategy` for the games in this repository. Every code block below is real, verified code: assembled into one file, it compiles against `common` as-is, beats the baselines, and plays a perfect-play draw against the depth-9 minimax — from both seats.

The guide targets plain tic-tac-toe because it is small enough to check the result against a known truth (the game is a draw), but nothing in the implementation is specific to it: the finished strategy is generic over the `Game` trait and runs unchanged on connect-four or ultimate tic-tac-toe.

1. Why MCTS when minimax already solves tic-tac-toe?

It does — depth 9 covers the whole game and `evaluate()` is never even called. Tic-tac-toe is the *training ground*, not the motivation. The motivation is what happened with ultimate tic-tac-toe: minimax beyond a few plies needs a hand-written evaluation function, and the search is blind to everything past its fixed horizon.

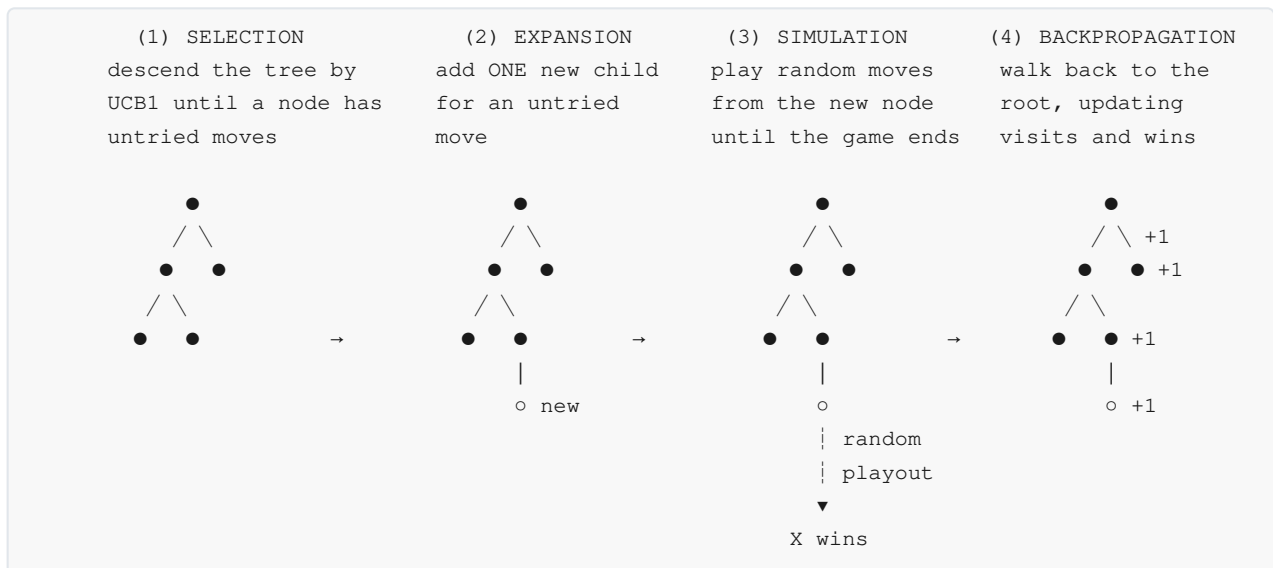
MCTS makes the opposite trade-offs:

	Minimax (alpha-beta)	MCTS (UCT)
Evaluation function	Required at the depth horizon	None — random playouts to the end
Horizon	Hard cutoff at <code>max_depth</code>	None; promising lines grow deeper
Budget	Whole search or nothing	Anytime — stop after N iterations or a time limit
Tree shape	Uniform depth, pruned width	Asymmetric — grows toward good moves
Result quality	Exact within horizon	Statistical, converges to optimal
Worst enemy	No heuristic / huge branching	Shallow tactical traps

The core idea: instead of *evaluating* a position, **sample** it. Play many fast random games from it; the fraction won is an estimate of how good the position is. A clever policy (UCB1) decides which positions deserve more samples, and the sample statistics themselves form the search tree.

2. The algorithm at a glance

One MCTS *iteration* has four phases, repeated thousands of times:



After the budget is spent, the move played is the root child with the most visits. That's the whole algorithm — the rest of this guide is making each phase precise and getting the one genuinely tricky part right (whose perspective a node's statistics are counted from).

3. Step 1 — Groundwork: what the `Game` trait already provides

MCTS needs five things from a game, all already on the trait:

- `clone()` — each iteration works on a scratch copy of the root position,
- `get_possible_moves()` — to expand and to roll out,
- `apply_move()` — to descend (no `undo_move` needed, unlike minimax),
- `is_finished()` / `get_winner()` — to stop and score a playout,
- `get_current_player()` — to know who a move belongs to.

The only thing missing is randomness. Rather than pulling in a crate, a seeded xorshift is five lines and makes every search **reproducible** — which you will appreciate the first time a test fails:

```

use common::search::Strategy;
use common::{Game, GameError, Player};

/// Tiny deterministic RNG (xorshift64*) – enough for rollouts, zero deps,
/// and a fixed seed makes every search reproducible.
struct Rng(u64);

impl Rng {
    fn next(&mut self) -> u64 {
        self.0 ^= self.0 << 13;
        self.0 ^= self.0 >> 7;
        self.0 ^= self.0 << 17;
        self.0
    }

    /// Uniform-ish index in `0..n` (bias is irrelevant at these sizes).
    fn below(&mut self, n: usize) -> usize {
        (self.next() % n as u64) as usize
    }
}

```

4. Step 2 — The tree as an arena

A search tree in Rust built from `Box / Rc<RefCell<...>>` nodes fights the borrow checker at every step (backpropagation walks *up* the tree, so nodes need parent pointers). The standard solution is an **arena**: all nodes live in one `Vec<Node>`, and "pointers" are plain `usize` indices. Cheap, cache-friendly, and the borrow checker is happy because there is only ever one owner — the `Vec`.

```

/// One node of the search tree, indexed into an arena `Vec<Node<G>>`.
struct Node<G: Game> {
    parent: Option<usize>,
    /// The move that led from `parent` to this node (`None` at the root).
    chosen_move: Option<G::Move>,
    /// Who made `chosen_move` — `wins` is from this player's perspective.
    mover: Option<G::PlayerMask>,
    children: Vec<usize>,
    /// Legal moves not yet expanded into children. Empty for terminal
/// positions, even if the game would still report legal moves.
    untried_moves: Vec<G::Move>,
    visits: f64,
    wins: f64,
}

impl<G: Game> Node<G> {
    fn new(
        parent: Option<usize>,
        chosen_move: Option<G::Move>,
        mover: Option<G::PlayerMask>,
        state: &G,
    ) -> Self {
        Node {
            parent,
            chosen_move,
            mover,
            children: Vec::new(),
            untried_moves: if state.is_finished() {
                Vec::new()
            } else {
                state.get_possible_moves().collect()
            },
            visits: 0.0,
            wins: 0.0,
        }
    }
}

```

Two fields deserve a closer look.

untried_moves is the expansion frontier: a node is "fully expanded" when it is empty. The `is_finished()` guard in `new` is load-bearing — `TicTacToe::get_possible_moves()` happily reports the empty cells of a board that *already has a winner*. Without the guard, the search would expand children of finished games and keep playing into won positions. If you take one pitfall away from this guide, take the two in here: this one, and the perspective rule below.

mover is the player who *made* the move into this node — not the player to move *in* it. **wins** is counted from **mover**'s perspective. Why becomes clear in selection.

The strategy itself is just configuration plus the RNG:

```
pub struct Mcts {
    pub iterations: usize,
    pub exploration: f64,
    rng: Rng,
}

impl Mcts {
    pub fn new(iterations: usize) -> Self {
        Mcts {
            iterations,
            exploration: std::f64::consts::SQRT_2,
            rng: Rng(0x9E37_79B9_7F4A_7C15),
        }
    }
}
```

5. Step 3 — Selection: UCB1

Selection descends from the root, repeatedly choosing among a node's children, until it reaches a node that still has untried moves (or is terminal). The choice is the heart of MCTS: it must balance

- **exploitation** — revisit children that have won a lot, to sharpen their statistics and build the tree deeper behind them, and
- **exploration** — occasionally revisit unpromising children, because their bad average might just be bad luck from few samples.

UCB1 ("Upper Confidence Bound") resolves the dilemma with one formula. Pick the child maximizing:

$$\text{UCB1} = \underbrace{\frac{\text{wins_child}}{\text{visits_child}}}_{\text{exploitation}} + c \cdot \sqrt{\underbrace{\frac{\ln(\text{visits_parent})}{\text{visits_child}}}_{\text{exploration}}}$$

The first term is the child's observed win rate. The second grows for children visited rarely relative to their parent — and shrinks again as they accumulate visits. Every child is guaranteed to be revisited eventually (the logarithm never stops growing), but exponentially less often if it keeps disappointing. The theoretical sweet spot for c with rewards in $[0, 1]$ is $\sqrt{2}$; treat it as a tunable.

Here is where the `mover` perspective pays off: the win rate used to choose among children must be from the viewpoint of **the player making the choice** — the player to move at the *parent*. That player is exactly the one who makes the move *into* each child, i.e. each child's `mover`. Storing wins from the mover's perspective makes selection a plain maximization at every level, with no sign-flipping — the MCTS analogue of negamax.

```

impl Mcts {
    /// The child of `parent` maximizing UCB1:
    /// `wins/visits + c * sqrt(ln(parent visits) / visits)`.
    fn select_child<G: Game>(&self, nodes: &[Node<G>], parent: usize) -> usize {
        let ln_parent = nodes[parent].visits.ln();

        let ucb = |i: usize| {
            let n = &nodes[i];
            n.wins / n.visits + self.exploration * (ln_parent / n.visits).sqrt()
        };

        *nodes[parent]
            .children
            .iter()
            .max_by(|&a, &b| ucb(a).total_cmp(&ucb(b)))
            .expect("select_child called on a node with children")
    }
}

```

No division-by-zero check is needed: a child is created in the same iteration that backpropagates through it, so every child has `visits >= 1` by the time selection can see it. (`total_cmp` is the no-`NaN`-surprises way to order floats.)

6. Step 4 — Expansion

When selection stops at a node with untried moves, exactly **one** of them (chosen at random — `swap_remove` is $O(1)$ and order doesn't matter) becomes a new child. Growing one node per iteration keeps the tree exactly as large as the number of iterations spent, concentrated where selection actually goes:

```

// 2. Expansion: add one random untried move as a new child.
let untried = nodes[node].untried_moves.len();
if untried > 0 {
    let mv = nodes[node].untried_moves.swap_remove(self.rng.below(untried));
    let mover = state.get_current_player();
    state.apply_move(mv);

    let child = Node::new(Some(node), Some(mv), Some(mover), &state);
    nodes.push(child);
    let child_index = nodes.len() - 1;
    nodes[node].children.push(child_index);
    node = child_index;
}

```

Note the order: `get_current_player()` is read **before** `apply_move` (which flips it) — that's the `mover` recorded in the child. The `if` falls through for terminal nodes (their `untried_moves` is empty by construction), in which case the "playout" below ends immediately and just reports the actual result.

7. Step 5 — Simulation

From the freshly expanded position, play uniformly random legal moves until the game ends, and report the winner. This is the "Monte Carlo" part — the playout is deliberately dumb and therefore fast; truth emerges from volume, not from cleverness:

```
impl Mcts {  
    /// Play uniformly random moves until the game ends.  
    fn rollout<G: Game>(&mut self, state: &mut G) -> Option<G::PlayerMask> {  
        let mut moves: Vec<G::Move> = Vec::new();  
  
        while !state.is_finished() {  
            moves.clear();  
            moves.extend(state.get_possible_moves());  
            state.apply_move(moves[self.rng.below(moves.len())]);  
        }  
  
        state.get_winner()  
    }  
}
```

The buffer is reused across the loop to avoid an allocation per move. In bigger games this function is the hot spot — "heavy playouts" (adding light tactical rules, like *take an immediate win if one exists*) trade speed per playout for signal per playout, and are the first thing to experiment with after the plain version works.

8. Step 6 — Backpropagation

The playout result must now update every node on the path from the expanded node back to the root. This is where most MCTS implementations go wrong on the first attempt, because consecutive nodes on that path belong to **opposite players**: a playout X wins is a success for every node whose `mover` is X and a failure for every node whose `mover` is O — the same result alternates meaning on the way up.

Because each node stores `wins` from its own `mover`'s perspective, the update stays local and simple — compare the playout winner against each node's own `mover`:

```
// 4. Backpropagation: score each node for its own mover.  
let mut current = Some(node);  
while let Some(i) = current {  
    nodes[i].visits += 1.0;  
    nodes[i].wins += match (&winner, &nodes[i].mover) {  
        (Some(w), Some(m)) if w == m => 1.0,  
        (Some(_), _) => 0.0,  
        (None, _) => 0.5,  
    };  
    current = nodes[i].parent;  
}
```

Draws score **0.5, not 0**. In tic-tac-toe this matters enormously: the game *is* a draw under perfect play, so an agent that counts draws as losses considers every correct line worthless and plays desperate, losing moves instead. Symptom of getting either of these wrong: the bot actively walks into defeats — selection is maximizing its opponent's success.

9. Step 7 — Putting it together as a **Strategy**

The main loop is the four phases in sequence, on a scratch clone of the position. Afterwards, the move played is the **most visited** root child — not the one with the best win rate. A 90% rate over 10 visits is noise; 60% over 2,000 visits is knowledge. Visit counts only grow large where UCB1 kept selecting, so they are the trustworthy signal ("robust child" in the literature):


```

impl<G: Game> Strategy<G> for Mcts {
    fn compute_move(&mut self, game: &G) -> Result<G::Move, GameError> {
        let mut nodes: Vec<Node<G>> = vec![Node::new(None, None, None, game)];

        for _ in 0..self.iterations {
            let mut state = game.clone();
            let mut node = 0;

            // 1. Selection: descend while fully expanded and not terminal.
            while nodes[node].untried_moves.is_empty() && !nodes[node].children.is_empty() {
                node = self.select_child(&nodes, node);
                let mv = nodes[node].chosen_move.expect("non-root has a move");
                state.apply_move(mv);
            }

            // 2. Expansion: add one random untried move as a new child.
            let untried = nodes[node].untried_moves.len();
            if untried > 0 {
                let mv = nodes[node].untried_moves.swap_remove(self.rng.below(untried));
                let mover = state.get_current_player();
                state.apply_move(mv);

                let child = Node::new(Some(node), Some(mv), Some(mover), &state);
                nodes.push(child);
                let child_index = nodes.len() - 1;
                nodes[node].children.push(child_index);
                node = child_index;
            }

            // 3. Simulation: random playout from the new position.
            let winner = self.rollout(&mut state);

            // 4. Backpropagation: score each node for its own mover.
            let mut current = Some(node);
            while let Some(i) = current {
                nodes[i].visits += 1.0;
                nodes[i].wins += match (&winner, &nodes[i].mover) {
                    (Some(w), Some(m)) if w == m => 1.0,
                    (Some(_), _) => 0.0,
                    (None, _) => 0.5,
                };
                current = nodes[i].parent;
            }
        }

        // The robust child: most visited, not best mean - visit counts are
        // the statistically reliable signal.
        nodes[0]
            .children
            .iter()
            .max_by(|&a, &b| nodes[a].visits.total_cmp(&nodes[b].visits))
            .and_then(|&best| nodes[best].chosen_move)
            .ok_or(GameError::NoMovesAvailable)
    }
}

```

Because `Strategy` is object-safe and generic over the game, this is a complete drop-in: no registration, no changes to `common`, and it can play against any existing strategy through `Competition`.

```
use common::Competition;
use common::search::baseline::HumanPlayer;
use tic_tac_toe::TicTacToe;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut competition = Competition::new(TicTacToe::new(), Mcts::new(10_000), HumanPlayer);
    competition.start(true)?;
    Ok(())
}
```

10. Step 8 — Testing it

Tic-tac-toe's killer feature as a training ground: the truth is known. The game is a draw under perfect play, and the repository already has a perfect player — `Minimax::new(9)`. That makes the strongest possible assertion cheap: minimax never loses, so *any* decisive result in an MCTS-vs-minimax game would be an MCTS loss. A draw proves MCTS made no losing mistake.

```

use common::search::baseline::FirstPossibleMove;
use common::search::minimax::Minimax;
use common::Competition;
use tic_tac_toe::{PlayerMask, TicTacToe};

#[test]
fn mcts_beats_first_possible_move() {
    let mut competition =
        Competition::new(TicTacToe::new(), Mcts::new(10_000), FirstPossibleMove);
    competition.start(false).unwrap();
    assert_eq!(competition.game.get_winner(), Some(PlayerMask::X));
}

#[test]
fn mcts_draws_against_minimax() {
    // Minimax plays perfectly, so it never loses; a draw means MCTS made
    // no losing mistake either.
    let mut competition =
        Competition::new(TicTacToe::new(), Mcts::new(10_000), Minimax::new(9));
    competition.start(false).unwrap();
    assert_eq!(competition.game.get_winner(), None);

    let mut competition =
        Competition::new(TicTacToe::new(), Minimax::new(9), Mcts::new(10_000));
    competition.start(false).unwrap();
    assert_eq!(competition.game.get_winner(), None);
}

#[test]
fn mcts_opening_is_strong() {
    // 10k iterations must find that only corners/center don't concede an
    // edge in tic-tac-toe's opening.
    const CORNERS_AND_CENTER: u16 = 0b1_0101_0101;
    let game = TicTacToe::new();
    let mv = Mcts::new(10_000).compute_move(&game).unwrap();
    assert!((1u16 << mv) & CORNERS_AND_CENTER > 0, "got {mv}");
}

```

All three pass in a few hundredths of a second in release mode — and because the RNG is seeded, they pass *deterministically*. With an OS-seeded RNG the draw test would be a coin-flip per CI run at low iteration counts; determinism turns "usually strong" into a regression test.

11. Step 9 — Tuning and the pitfall checklist

Iterations. 10,000 is generous for tic-tac-toe (the full game tree has only ~5,500 distinct positions). The interesting regime is low: at ~100 iterations MCTS still blunders tactically — statistical knowledge needs samples. For bigger games, replace the `for` loop with a time budget; nothing else changes — MCTS is anytime by construction:

```

let deadline = std::time::Instant::now() + std::time::Duration::from_millis(95);
while std::time::Instant::now() < deadline {
    // ... one iteration ...
}

```

Exploration constant. $\sqrt{2}$ is the principled default for rewards in `[0, 1]`. Lower values commit harder to early favorites (risky with few iterations), higher values spread thinner. Tune it only with a fixed-seed A/B setup — its effect is smaller than rollout quality.

The checklist — the four bugs that account for virtually every broken MCTS, all invisible until the bot mysteriously plays badly:

1. **Perspective:** a node's `wins` must be from its `mover`'s viewpoint, compared against *that node's own mover* during backpropagation. Wrong sign = the bot optimizes for its opponent.
2. **Draws are 0.5.** Scoring them 0 makes a drawn game (like tic-tac-toe) look uniformly hopeless, and the bot stops defending.
3. **Terminal nodes must not expand.** If the game still reports legal moves after a win (tic-tac-toe does), guard `untried_moves` with `is_finished()`.
4. **mover is read before `apply_move`** — applying flips the current player; reading it afterwards shifts every node's statistics one ply.

12. Beyond tic-tac-toe

The implementation above is generic over `Game`, so it already runs on ultimate tic-tac-toe —

`Competition::new(UltimateTicTacToe::new(), Mcts::new(100_000), HumanPlayer)` works today. That matchup is exactly where MCTS earns its keep: no hand-tuned `evaluate()`, no horizon, and the asymmetric tree handles the wildly varying branching factor (9–81) naturally. MCTS is the standard approach at the top of CodinGame's UTTT leaderboard.

Worthwhile upgrades, roughly in order of value:

- **Tree reuse:** after the opponent moves, the relevant subtree already exists — re-root instead of starting cold (the arena makes this a copy).
- **Heavy playouts:** "win if you can, block if you must" inside `rollout` buys a large strength jump per playout in tactical games.
- **First Play Urgency / progressive bias:** better behavior at low visit counts than vanilla UCB1.
- **RAVE / AMAF:** share playout statistics across sibling moves — strong in games where a good move is good regardless of order.
- **PUCT:** replace the exploration term with a learned move prior — the AlphaZero family is this same loop with the random rollout swapped for a neural network's value head.

The four-phase skeleton never changes. Once it is correct — perspective, draws, terminals, mover timing — every one of these is a local edit.